Project Title:
Deterministic Multi-Task Embedded Control Framework (STM32 + FreeRTOS)

Goal:

Build a real-time embedded firmware system that demonstrates **deterministic timing**, **static memory discipline**, and **structured RTOS task orchestration** — the core skills required in modern embedded C++ roles.

Project Objective

Design and implement a real-time embedded firmware framework that:

- Uses hardware timers for clock-driven ADC sampling
- Executes prioritized RTOS tasks with deterministic behavior
- Enforces static memory allocation (no dynamic heap usage)
- Implements modular hardware abstraction and clear task boundaries
- Operates continuously under watchdog supervision

Target Hardware

- STM32 Nucleo board
- One analog input — 10kΩ potentiometer on PA1 (header A1), simulating a sensor input
- One digital output — AEDIKO 5V optocoupler relay module (high-trigger) on PB0 (header A3). Red LED wired to relay NO terminal as fault indicator.
- UART telemetry — USART2 via ST-Link virtual COM port at 115200 baud. Onboard green LED LD2 on PA5 as normal-state indicator.

Software Layers

- **HAL Layer:** ADC, Timer, GPIO, UART
- **Driver Layer:** Circular buffer, ADC driver, relay driver
- **Application Layer:** Tasks, control logic, logging

Data Flow

1. Timer ISR → ADC sample
2. Sample → static ring buffer
3. Task A pulls samples → computes average
4. Task B reacts to threshold events
5. Task C logs system state

Key Features

1. Deterministic Timer-driven ADC Sampling

   Demonstrates:
   - Hardware timer configuration
   - Interrupt-driven design
   - Deterministic 1kHz sampling
   - ISR minimalism discipline

   Specification:
   - Hardware timer configured at 1 kHz
   - Timer interrupt triggers ADC conversion
   - ISR reads ADC register directly
   - Sample stored in statically allocated circular buffer
   - ISR execution time < 10% of timer period
   - No blocking or complex logic inside ISR

   Constraint:
   ISR must remain bounded and non-blocking.

2. RTOS Multi-Task Scheduling

   Demonstrates:
   - FreeRTOS integration
   - Task prioritization strategy
   - Queue-based communication
   - Separation of concerns

   Specification:
   Task A: Data Processing (med priority)
   - Pulls samples from queue

- Computes rolling average
- Detects threshold events

Task B: Control Logic (high priority)
- Responds to threshold violations
- Toggles digital output deterministically

Task C: UART Logging (low priority)
- Outputs telemetry
- Logs system state

- Use FreeRTOS queue for data transfer
- No global shared variables

3. Static Memory & Safety Discipline
   Demonstrates:
   - Embedded memory awareness
   - Deterministic allocation strategy
   - Long-term stability thinking

   Specification:
   - No malloc / new
   - No STL containers
   - All buffers fixed-size at compile time
   - Explicit ownership model per module
   - Independent watchdog enabled
   - System must run continuously for 24h without crash

System Architecture

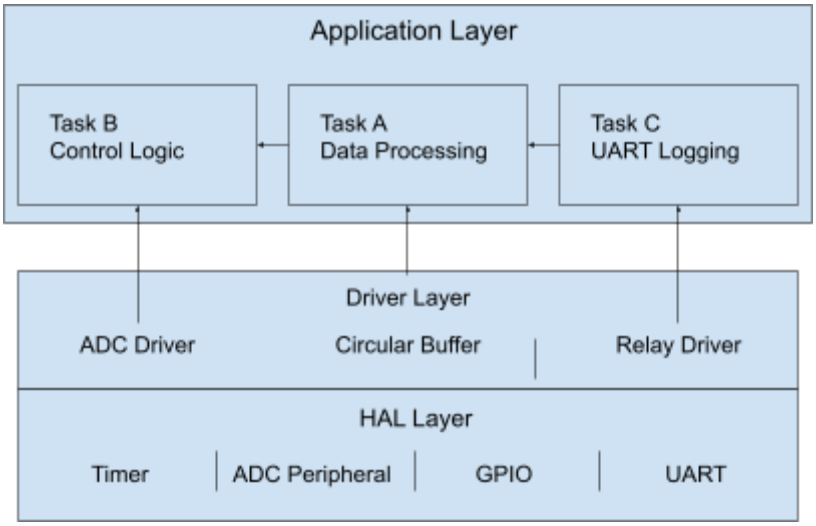

Fig. 1: System Block Diagram

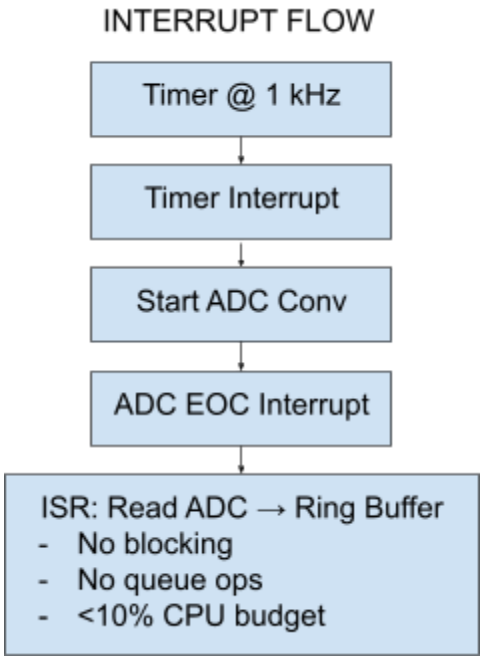

Fig. 2: Interrupt Flow Diagram

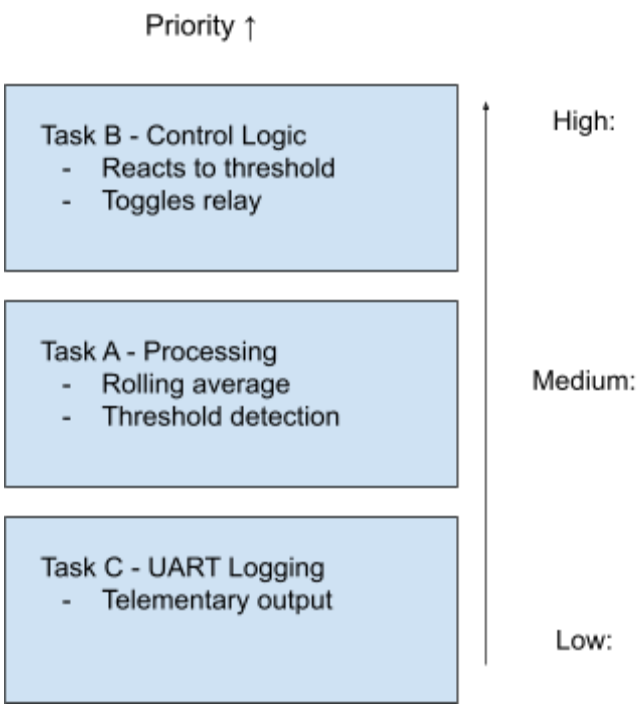

Fig. 3: Task Priority Diagram

Region	Size	Contents	Notes
Flash	128 KB	Firmware code, const tables	No dynamic allocation
RAM (Static)	8 KB	Task stacks, ring buffer, queues	All compile-time allocated
Ring Buffer	256 bytes	128 samples × 2 bytes	Owned by ADC driver
Task Stacks	1 KB each	Fixed stacks	No overflow allowed
Watchdog	32 bytes	WDG registers	Kicked by Task B

Table 1: Memory Allocation Map

Project Rules:

- No dynamic memory allocation
- No delay()
- No polling loops for timing
- ISR must remain bounded
- All modules separated into:
    - HAL layer
    - Driver layer
    - Application layer

Project Summary

This project implements a deterministic multi-task embedded control system using STM32 and FreeRTOS. It demonstrates timer-driven ADC sampling, priority-based task scheduling, static memory discipline, and watchdog-supervised long-term stability."

Feature Definition

Success Criteria

- ADC sampling jitter < 5%
- ISR < 10% of timer period
- No missed watchdog kicks
- No heap usage
- System runs 24 hours without reset

Risks & Mitigations

- Risk: ISR too long → Mitigation: minimize logic
- Risk: Queue overflow → Mitigation: bounded processing time
- Risk: UART blocking → Mitigation: low-priority task + ring buffer

Design Rationale

- Why 1 kHz Sampling?
    - 1 kHz sampling was selected to simulate a real industrial control loop frequency commonly used in power electronics and monitoring systems. It provides sufficient temporal resolution to detect fast-changing input signals while remaining computationally lightweight for an STM32F401RE running FreeRTOS. At 1 ms intervals, timing drift and jitter become measurable and meaningful for deterministic validation.
- Why no dynamic allocation?
    - Dynamic memory allocation is excluded to eliminate heap fragmentation risk and non-deterministic allocation latency. In embedded systems, especially real-time environments, bounded memory behavior is critical. All buffers, task stacks, and queues are statically allocated at compile time to guarantee deterministic memory usage and long-term runtime stability.
- Why Control Logic is highest priority?

    - The Control Logic task directly influences physical outputs (relay state) and therefore represents the system's real-time response path. It must preempt lower-priority tasks to ensure output state changes occur within bounded latency. Logging and processing are secondary functions and must never delay control execution.
- Why no queue ops in ISR?
    - Queue operations inside an ISR increase execution time and introduce potential blocking or priority inversion risks. The ISR is intentionally minimized to:
        - Read ADC register
        - Store value in static ring buffer
        - Exit
    - All inter-task communication occurs outside interrupt context to maintain bounded and predictable ISR timing.
- Why watchdog tied to Task B?
    - Task B represents the final control decision layer. If Task B fails to execute correctly, the system's output behavior cannot be trusted. Therefore, the watchdog is refreshed only after successful completion of Task B's execution cycle. This ensures that stalled processing, deadlocks, or task starvation will trigger system recovery.

Fault Handling Strategy

- ADC Stuck Value
    - If identical ADC readings persist beyond a defined threshold window, the system flags a sensor fault. Task B forces relay to safe state and logs fault condition via UART.
- Queue Overflow
    - If the queue reaches capacity, oldest unprocessed samples are discarded (overwrite strategy) to prioritize real-time behavior over historical completeness. Overflow events increment an error counter.
- Missed Execution Window
    - If Task B does not execute within expected timing bounds, a timing violation flag is raised. Repeated violations result in watchdog-triggered system reset.
- Task Starvation
    - FreeRTOS stack high-water mark monitoring is enabled. If stack usage approaches defined limit, system logs warning and enters safe state prior to reset.
- Watchdog Reset Behavior

- When watchdog triggers:
  - MCU performs hardware reset.
  - Relay defaults to safe (OFF) state via hardware-defined startup configuration.
  - Boot sequence logs reset cause to UART.
  - System resumes normal operation.
- Fail-safe behavior always defaults output to non-energized state.

Timing Budget

Assume STM32F401RE @ 84 MHz.

1 kHz timer = 1 ms period.

You want ISR < 10% → < 100 µs.

Component	Max Execution Time	Budget %
Timer ISR	< 50 µs	< 5%
Task A	< 150 µs	Bounded
Task B	< 100 µs	Highest priority
Task C	Non-critical	Best effort

Table 2: Timing Budget

Business-Level Explanation

This system simulates a production-grade industrial control loop where deterministic timing and predictable memory behavior are critical. It demonstrates separation of data acquisition, control decision-making, and system monitoring to prevent functional interference between subsystems. The architecture prioritizes reliability, bounded execution time, and fault recovery — principles required in safety-critical and infrastructure-grade embedded systems.

Hardware Section

Selected MCU: STM32F401RE

- Core: ARM Cortex-M4 @ 84 MHz
- Flash: 512 KB
- SRAM: 96 KB
- 12-bit ADC with multiple channels
- Multiple hardware timers

Sensor Selection Note — ACS712 Incompatibility

The ACS712 Hall-effect current sensor was evaluated as the primary analog input but rejected due to a voltage incompatibility. The ACS712 requires a 5V supply and its output swings proportionally up to 5V under load. The STM32F401RE ADC inputs are limited to 3.3V maximum — a direct connection risks permanent damage to the MCU.

A 10kΩ potentiometer on PA1 (ADC Channel 1) was substituted as the analog input, providing a clean 0–3.3V range across the full ADC scale (0–4095 counts). The firmware architecture is sensor-agnostic — replacing the potentiometer with a production sensor requires only a channel and pin change. For a real current sensing deployment, the INA219 (I2C, 3.3V native) is the recommended upgrade path.

Relay Module — Polarity & Pin Mapping

The AEDIKO 5V optocoupler relay module is a high-trigger design: GPIO HIGH energizes the coil, GPIO LOW releases it. The relay IN pin is wired to PB0, which maps to Arduino header pin A3 (not D3). The startup default for PB0 is set HIGH in MX_GPIO_Init to ensure the relay remains de-energized during boot, before the RTOS scheduler starts Task B.

Why This MCU?

The STM32F401RE provides sufficient clock speed and SRAM to support FreeRTOS task stacks, static buffers, and queue structures without resource pressure. The 84 MHz clock ensures ISR and task execution remain well within timing budgets for 1 kHz

Why It Easily Supports 1 kHz Deterministic Sampling

At 84 MHz:
1 ms = 84,000 clock cycles.
Even a 50 µs ISR uses only ~4,200 cycles.
This leaves significant headroom for task scheduling and control computation while maintaining bounded timing guarantees.

Implementation Status

The project was fully implemented and verified on an STM32F401RE Nucleo board using a potentiometer as the analog input (simulating a current sensor) and the onboard LED LD2 as the digital output (simulating a relay). The firmware architecture is hardware-agnostic — swapping to a real current sensor and relay requires only a pin and channel change, with no architectural modifications.

Completed Features

- **TIM2 hardware timer at 1 kHz** — Prescaler 83, Period 999 at 84 MHz. Timer ISR triggers ADC conversion and writes samples to the ring buffer. ISR is minimal and non-blocking.
- **Static ring buffer (64 samples)** — Fixed-size, power-of-2 circular buffer with no dynamic allocation. ISR writes, Task A reads. Overflow strategy discards oldest sample to preserve real-time behavior.
- **Task A — Data Processing (Normal priority)** Drains ring buffer, computes 16-sample rolling average, detects stuck sensor values (10 consecutive identical readings triggers safe state), pushes averages to FreeRTOS queue.
- **Task B — Control Logic (High priority)** Reads queue, controls LED/relay output against 2048 threshold, kicks IWDG watchdog every cycle. Watchdog timeout ~2 seconds — system resets if Task B stalls.
- **Task C — UART Logging (Low priority)** Outputs telemetry every 500 ms via USART2 at 115200 baud. Format: ADC:[value] OVFL:[faults] STK:[stack headroom]. Stack high-water mark monitored in real time.
- **IWDG Watchdog** — Hardware independent watchdog enabled and tied exclusively to Task B. Prescaler 16, reload 4095, timeout ~2 seconds.

Verified Outputs

Live UART telemetry confirmed ADC values tracking potentiometer position across the full 0–4095 range. Green LED (LD2) and relay transitions verified at the 2048 mid-scale threshold: pot above halfway energizes LD2 and de-energizes the relay; pot below halfway de-energizes LD2 and energizes the relay (red fault LED activates via relay NO terminal). Stack headroom stable at 130 words across sustained operation. Overflow counter remained 0 under normal operation. Stuck-value sentinel (0xDEAD) triggers correctly on disconnected or frozen input. Watchdog reset confirmed when Task B execution was interrupted.

Hardware Upgrade Path

To deploy on production hardware, change ADC_CHANNEL_1 to the current sensor input channel, update the GPIO pin for the relay driver, and adjust ADC_THRESHOLD to match the

current trip point in ADC counts. All task logic, timing, memory discipline, and watchdog behavior remain unchanged.

Appendix A — Diagrams

A1. System Block Diagram

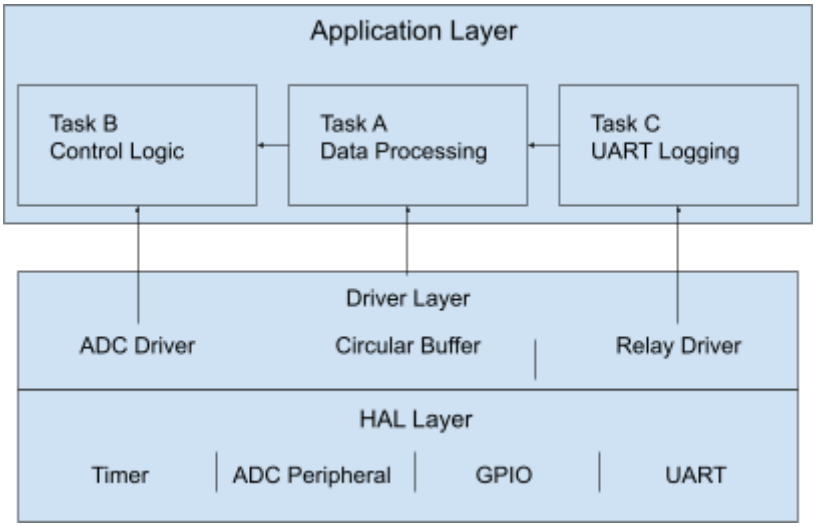

Fig. 1: System Block Diagram

A2. Interrupt Flow Diagram

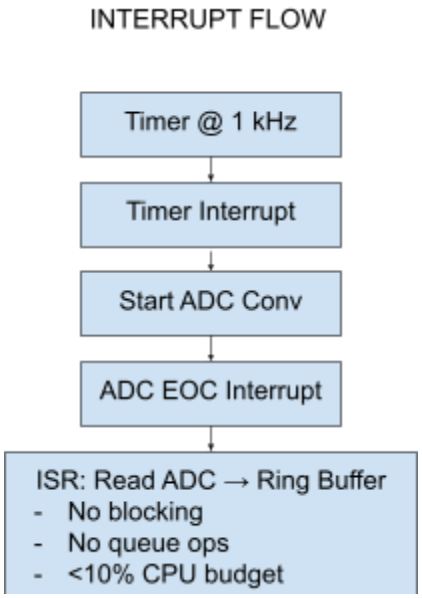

Fig. 2: Interrupt Flow Diagram

A3. Task Priority Diagram

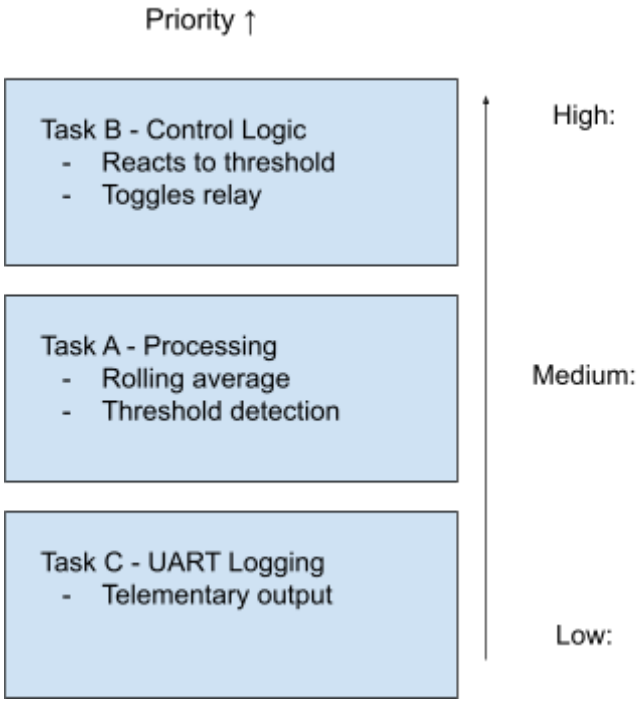

Fig. 3: Task Priority Diagram